



PIMPRI CHINCHWAD EDUCATION TRUST'S.  
**PIMPRI CHINCHWAD COLLEGE OF ENGINEERING**  
(An Autonomous Institute)  
Affiliated to Savitribai Phule Pune University (SPPU)  
ISO 21001:2018 Certified by TUV SUD

**T.Y. B. TECH**

**Branch:** Computer Engineering

**Year:** 2025 – 26

**Semester:** 6

**Date:** 1/4/2026

### **Assignment 6**

**Assignment Title:** Create Online appointment booking application / E-commerce portal for used items sales like car, bike etc. or similar type application using Node.js and Express use any NoSQL data set like MongoDB if required.

**Name:** Trushita Sathe

**PRN:** 124B2B024

**DIV:** C

#### **Objectives:**

- To develop a dynamic Online Appointment Booking system using Node.js and Express
- To create a user-friendly form for booking CA appointments
- To implement proper form validations (required fields, phone number restriction, future date/time validation). Prevent appointment clashes . Delete or cancel appointments.
- To integrate frontend with backend using Fetch API
- To store appointment data in MongoDB database

#### **Outcomes:**

- A fully functional Online Appointment Booking system
- Successful submission of appointment data to backend
- Appointment details stored in MongoDB database
- Automatic display of booked appointments under respective Chartered Accountant
- Prevention of past date/time bookings
- Improved user experience with structured validations and organized display

### **Theory Concept in brief:**

The Online CA Appointment Booking System is built using a full-stack JavaScript architecture. The system allows users to book appointments with Chartered Accountants. It validates user inputs, prevents invalid bookings, and stores appointment data in a NoSQL database. The backend handles API requests while the frontend dynamically updates the UI without page reload.

### **Technology Stack Used**

- **HTML & CSS**

Used to design and structure the user interface, including the booking form and sidebar display.

- **JavaScript (Frontend)**

Used to handle form submission, validations, dynamic updates, and API communication using Fetch API.

- **Node.js**

Server-side JavaScript runtime environment used to run the backend application.

- **Express.js**

Used to create REST APIs for handling appointment booking, retrieving appointments, and managing data.

- **MongoDB**

Used as NoSQL database to store appointment records.

- **Mongoose**

Used as ODM (Object Data Modeling) tool to define schema and interact with MongoDB.

### **Dataset used:**

#### **MongoDB**

**Used as database to store records.**

#### **MongoDB**

Used as database to store appointment booking records.

Each record contains:

name  
email  
phone  
caName  
date  
time

purpose  
createdAt

The database stores structured appointment information and allows retrieval and grouping of data based on Chartered Accountant.

### Program code:

#### Appointment.js

```
const mongoose = require("mongoose");

const appointmentSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true },
  phone: { type: String, required: true },
  caName: { type: String, required: true },
  date: { type: String, required: true },
  time: { type: String, required: true },
  purpose: { type: String, required: true }
}, { timestamps: true });

module.exports = mongoose.model("Appointment", appointmentSchema);
```

#### index.html

```
<!DOCTYPE html>

<html>
<head>
  <title>CA Appointment Booking</title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <link rel="stylesheet"
    href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.5.0/css/all.min.css"/>

  <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="layout">

  <div class="sidebar">
    <h2><i class="fa-solid fa-user-tie"></i> CAs</h2>

    <div class="ca-section">
```

```
<h3>CA Rajesh Sharma (<span id="count1">0</span>)</h3>
<div id="ca1Appointments"></div>
</div>

<div class="ca-section">
  <h3>CA Priya Mehta (<span id="count2">0</span>)</h3>
  <div id="ca2Appointments"></div>
</div>

<div class="ca-section">
  <h3>CA Amit Verma (<span id="count3">0</span>)</h3>
  <div id="ca3Appointments"></div>
</div>
</div>

<div class="main-content">
  <div class="form-container">
    <h1><i class="fa-solid fa-calendar-check"></i> Book Appointment</h1>

    <form id="appointmentForm">

      <label>Full Name</label>
      <input type="text" id="name" required>

      <label>Email Address</label>
      <input type="email" id="email" required>

      <label>Phone Number</label>
      <input type="text" id="phone" maxlength="10" required>

      <label>Select Chartered Accountant</label>
      <select id="caName" required>
        <option value="">-- Select CA --</option>
        <option value="CA Rajesh Sharma">CA Rajesh Sharma</option>
        <option value="CA Priya Mehta">CA Priya Mehta</option>
        <option value="CA Amit Verma">CA Amit Verma</option>
      </select>

      <label>Select Date</label>
      <input type="date" id="date" required>

      <label>Select Time</label>
      <input type="time" id="time" required>

      <label>Purpose of Meeting</label>
```

```

        <textarea id="purpose" required></textarea>

        <button type="submit">
            <i class="fa-solid fa-paper-plane"></i> Book Appointment
        </button>

    </form>
</div>
</div>
</div>

<script src="script.js"></script>
</body>
</html>

```

### Script.js

```

const form = document.getElementById("appointmentForm");

form.addEventListener("submit", async (e) => {
    e.preventDefault();

    const name = document.getElementById("name").value.trim();
    const email = document.getElementById("email").value.trim();
    const phone = document.getElementById("phone").value.trim();
    const caName = document.getElementById("caName").value;
    const date = document.getElementById("date").value;
    const time = document.getElementById("time").value;
    const purpose = document.getElementById("purpose").value.trim();

    if (!name || !email || !phone || !caName || !date || !time || !purpose) {
        alert("All fields are compulsory.");
        return;
    }

    if (!/^[0-9]{10}$/.test(phone)) {
        alert("Phone number must be exactly 10 digits.");
        return;
    }

    const selectedDateTime = new Date(`${date}T${time}`);

```

```

const now = new Date();

if (selectedDateTime <= now) {
    alert("Appointment must be scheduled for a future date and time.");
    return;
}

const appointment = { name, email, phone, caName, date, time, purpose };

const response = await fetch("/api/appointments", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(appointment)
});

const result = await response.json();

if (!response.ok) {
    alert(result.message);
    return;
}

alert("Appointment booked successfully!");
form.reset();
loadAppointments();
});

// DELETE
async function deleteAppointment(id) {
    await fetch(`/api/appointments/${id}`, {
        method: "DELETE"
    });
}

// LOAD
async function loadAppointments() {
    const response = await fetch("/api/appointments");
    const data = await response.json();

    document.getElementById("ca1Appointments").innerHTML = "";
    document.getElementById("ca2Appointments").innerHTML = "";
    document.getElementById("ca3Appointments").innerHTML = "";

    let count1 = 0, count2 = 0, count3 = 0;

```

```

const now = new Date();

data.forEach(app => {

  const appointmentDateTime = new Date(`${app.date}T${app.time}`);

  const dateObj = new Date(app.date);
  const formattedDate = dateObj.toLocaleDateString("en-GB", {
    day: "2-digit",
    month: "2-digit",
    year: "2-digit"
  });

  const timeObj = new Date(`1970-01-01T${app.time}`);
  const formattedTime = timeObj.toLocaleTimeString("en-US", {
    hour: "numeric",
    minute: "2-digit",
    hour12: true
  });

  const div = document.createElement("div");
  div.classList.add("appointment-item");

  // FUTURE → CANCEL
  if (appointmentDateTime > now) {
    div.innerHTML = `
      <strong>${app.name}</strong><br>
      Date: ${formattedDate}<br>
      Time: ${formattedTime}<br>
      <button onclick="cancelAppointment('${app._id}')">Cancel</button>
    `;
  }
  // PAST → MEETING DONE
  else {
    div.innerHTML = `
      <strong>${app.name}</strong><br>
      Date: ${formattedDate}<br>
      Time: ${formattedTime}<br>
      <button onclick="completeAppointment('${app._id}',
'${app.caName}')">Meeting Done</button>
    `;
  }

  if (app.caName === "CA Rajesh Sharma") {
    document.getElementById("ca1Appointments").appendChild(div);
  }
}

```

```

    }
    else if (app.caName === "CA Priya Mehta") {
        document.getElementById("ca2Appointments").appendChild(div);
    }
    else if (app.caName === "CA Amit Verma") {
        document.getElementById("ca3Appointments").appendChild(div);
    }
});

// Update counts
document.getElementById("count1").innerText = localStorage.getItem("count1")
|| 0;
document.getElementById("count2").innerText = localStorage.getItem("count2")
|| 0;
document.getElementById("count3").innerText = localStorage.getItem("count3")
|| 0;
}

// CANCEL
async function cancelAppointment(id) {
    await deleteAppointment(id);
    loadAppointments();
}

// MEETING DONE
async function completeAppointment(id, caName) {

    if (caName === "CA Rajesh Sharma") {
        let c = parseInt(localStorage.getItem("count1") || 0);
        localStorage.setItem("count1", ++c);
    }
    else if (caName === "CA Priya Mehta") {
        let c = parseInt(localStorage.getItem("count2") || 0);
        localStorage.setItem("count2", ++c);
    }
    else if (caName === "CA Amit Verma") {
        let c = parseInt(localStorage.getItem("count3") || 0);
        localStorage.setItem("count3", ++c);
    }

    await deleteAppointment(id);
    loadAppointments();
}

loadAppointments();

```

## style.css

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: Arial, Helvetica, sans-serif;
}

body {
  background: #f4f6fa;
}

/* Layout */
.layout {
  display: flex;
  min-height: 100vh;
}

/* Sidebar */
.sidebar {
  width: 320px;
  background: #1e3c72;
  color: white;
  padding: 20px;
  overflow-y: auto;
}

.sidebar h2 {
  margin-bottom: 20px;
}

.ca-section {
  margin-bottom: 25px;
}

.ca-section h3 {
  font-size: 16px;
  margin-bottom: 10px;
  border-bottom: 1px solid rgba(255,255,255,0.3);
  padding-bottom: 5px;
}
```

```
.appointment-item {
  background: white;
  color: black;
  padding: 8px;
  border-radius: 5px;
  margin-bottom: 8px;
  font-size: 13px;
}

/* Main Content */
.main-content {
  flex: 1;
  display: flex;
  justify-content: center;
  align-items: center;
}

.form-container {
  background: white;
  padding: 30px;
  width: 100%;
  max-width: 500px;
  border-radius: 8px;
  box-shadow: 0 5px 20px rgba(0,0,0,0.1);
}

.form-container h1 {
  margin-bottom: 20px;
  color: #1e3c72;
}

form input,
form select,
form textarea {
  width: 100%;
  padding: 10px;
  margin-bottom: 12px;
  border: 1px solid #ccc;
  border-radius: 5px;
}

form textarea {
  resize: none;
  height: 80px;
}
```

```

form button {
  width: 100%;
  padding: 10px;
  background: #1e3c72;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}

form button:hover {
  background: #16325c;
}

/* Responsive */
@media(max-width: 900px) {
  .layout {
    flex-direction: column;
  }

  .sidebar {
    width: 100%;
  }
}

label {
  font-weight: 600;
  font-size: 14px;
  margin-bottom: 5px;
  display: block;
}

form input,
form select,
form textarea {
  margin-bottom: 15px;
}

```

### Appointmentroute.js

```

const express = require("express");

const router = express.Router();
const Appointment = require("../models/Appointment");

```

```
// CREATE Appointment (WITH SLOT CHECK)
router.post("/", async (req, res) => {
  try {
    console.log("POST Body:", req.body);

    const { name, email, phone, caName, date, time, purpose } = req.body;

    // Check if same CA + date + time already exists
    const existing = await Appointment.findOne({
      caName: caName,
      date: date,
      time: time
    });

    if (existing) {
      return res.status(400).json({
        success: false,
        message: "Slot already booked for this CA"
      });
    }

    // Save new appointment
    const appointment = new Appointment({
      name,
      email,
      phone,
      caName,
      date,
      time,
      purpose
    });

    await appointment.save();

    res.status(200).json({
      success: true,
      message: "Appointment booked successfully"
    });

  } catch (error) {
    console.log("POST ERROR:", error);
    res.status(500).json({
      success: false,
      message: "Server error"
    });
  }
});
```

```

    }
  });

  // GET All Appointments
  router.get("/", async (req, res) => {
    try {
      const appointments = await Appointment.find();
      res.status(200).json(appointments);
    } catch (error) {
      console.log("GET ERROR:", error);
      res.status(500).json([]);
    }
  });

  // DELETE Appointment
  router.delete("/:id", async (req, res) => {
    try {
      await Appointment.findByIdAndDelete(req.params.id);
      res.status(200).json({ success: true });
    } catch (error) {
      console.log("DELETE ERROR:", error);
      res.status(500).json({ success: false });
    }
  });

  module.exports = router;

```

## server.js

```

const express = require("express");

const mongoose = require("mongoose");
const dotenv = require("dotenv");
const cors = require("cors");
const path = require("path");

dotenv.config();

const app = express();

app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));

```

```

mongoose.connect(process.env.MONGO_URI)
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));

app.use("/api/appointments", require("./routes/appointmentRoutes"));

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});

```

## Output screenshot:

The screenshot displays a web application interface with two main sections:

- Left Panel (CA Appointments):** A dark blue sidebar titled "CAs" lists three appointments:
  - CA Rajesh Sharma (1):** Arnab Abhijeet Sathe, Date: 24/03/26, Time: 11:30 AM, with a "Cancel" button.
  - CA Priya Mehta (2):** Janki Darandale, Date: 24/03/26, Time: 12:51 PM, with a "Cancel" button.
  - CA Amit Verma (1):** Janki Darandale, Date: 24/03/26, Time: 12:51 PM, with a "Cancel" button.
- Right Panel (Book Appointment):** A white form titled "Book Appointment" with the following fields:
  - Full Name:** Text input field.
  - Email Address:** Text input field.
  - Phone Number:** Text input field.
  - Select Chartered Accountant:** Dropdown menu with "-- Select CA --".
  - Select Date:** Date picker showing "mm/dd/yyyy".
  - Select Time:** Time picker showing "--:--".
  - Purpose of Meeting:** Text input field.
  - Submit Button:** A dark blue button labeled "Book Appointment" with a right-pointing arrow icon.

localhost:5000 says  
Phone number must be exactly 10 digits.  
OK

Full Name

Janki Darandale

Email Address

janki.darandale@gmail.com

Phone Number

78945612

Select Chartered Accountant

CA Rajesh Sharma

Select Date

02/28/2026

Select Time

08:00 AM

Purpose of Meeting

Discussion

Book Appointment

localhost:5000 says  
Appointment must be scheduled for a future date and time.  
OK

Full Name

Janki Darandale

Email Address

janki.darandale@gmail.com

Phone Number

7894561277

Select Chartered Accountant

CA Rajesh Sharma

Select Date

02/12/2026

Select Time

08:00 AM

Purpose of Meeting

Discussion

Book Appointment

